Experiment No-29 : Forward Chaining

Aim:

To write a Prolog program to implement Forward Chaining and incorporate the required queries.

Programming Code:

```
% Forward chaining: start from the known facts and apply the rules repeatedly
% to derive new facts until nothing new can be added.

:- dynamic fact/1.

% Initial (known) facts
initial(has_fur).
initial(gives_milk).
initial(eats_meat).
initial(has_claws).
initial(has_stripes).

% rule(ListOfConditions, Conclusion)
rule([has_fur, gives_milk], mammal).
rule([has_feathers, lays_eggs], bird).
rule([mammal, eats_meat], carnivore).
rule([carnivore, has_claws], predator).
rule([predator, has_stripes], tiger).

% Load the initial facts into the working memory
init :-
    retractall(fact(_)),
    forall(initial(F), assertz(fact(F))).

% All conditions of a rule are true
all_true(Conditions) :- forall(member(C, Conditions), fact(C)).

% Fire a rule whose conditions are true and whose conclusion is new
forward :-
    rule(Conditions, Conclusion),
    \+ fact(Conclusion),
    all_true(Conditions),
    assertz(fact(Conclusion)),
    format("Rule fired: ~w => ~w~n", [Conditions, Conclusion]),
    forward, !.
forward.

% Run forward chaining and print the final facts
run_forward :-
    init,
    forward,
    format("Final facts:~n"),
    forall(fact(F), format("  ~w~n", [F])).
```

OUTPUT:

```
$ swipl -q exp29.pl

?- init.
true.

?- fact(predator).
false.

?- run_forward.
Rule fired: [has_fur,gives_milk] => mammal
Rule fired: [mammal,eats_meat] => carnivore
Rule fired: [carnivore,has_claws] => predator
Rule fired: [predator,has_stripes] => tiger
Final facts:
  has_fur
  gives_milk
  eats_meat
  has_claws
  has_stripes
  mammal
  carnivore
  predator
  tiger
true.

?- fact(predator).
true.

?- fact(tiger).
true.

?- fact(bird).
false.

$
```

Result

Forward Chaining was implemented successfully using Prolog. Starting from the initial facts, the program fires the rules one by one and derives the new facts mammal, carnivore, predator and tiger. The rule for bird does not fire because its conditions are not satisfied.